



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Putri Angel Li
NPM	2428240091
Kelas	SI5B
Topik Tugas 1	Telekomunikasi: Paket Data
Entitas utama	data-plans
Tautan repositori	https://github.com/Putriangelli75/SI5B_Kuis1_Putri_Angel_Li
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Kamis, 8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Telekomunikasi: Paket Data
Entitas utama (tunggal)	data-plans
Alamat dasar	/data-plans
Field data	Id, namaPaket, operator, kuota, masaAktifHari, harga
Field wajib saat POST	namaPaket, operator, kuota, masaAktifHari, harga
Query string untuk filter	namaPaket, operator, kuota
Data awal	{ id: 1, namaPaket: 'Internet Bulanan 25GB', operator: 'Nusanet', kuotaGb: 25, masaAktifHari: 30, harga: 85000 }, { id: 2, namaPaket: 'Internet Bulanan 10GB', operator: 'Telkomsel', kuotaGb: 10, masaAktifHari: 30, harga: 50000 }, { id: 3, namaPaket: 'Internet Bulanan 5GB', operator: 'Axis', kuotaGb: 5, masaAktifHari: 30, harga: 30000 }
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[data-plans]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[data-plans]/1	Tidak	200	Ambil satu data
3	POST	/[data-plans]	Ya	201	Tambah data
4	PUT	/[data-plans]/3	Ya	200	Ubah data
5	DELETE	/[data-plans]/3	Ya	204	Hapus data

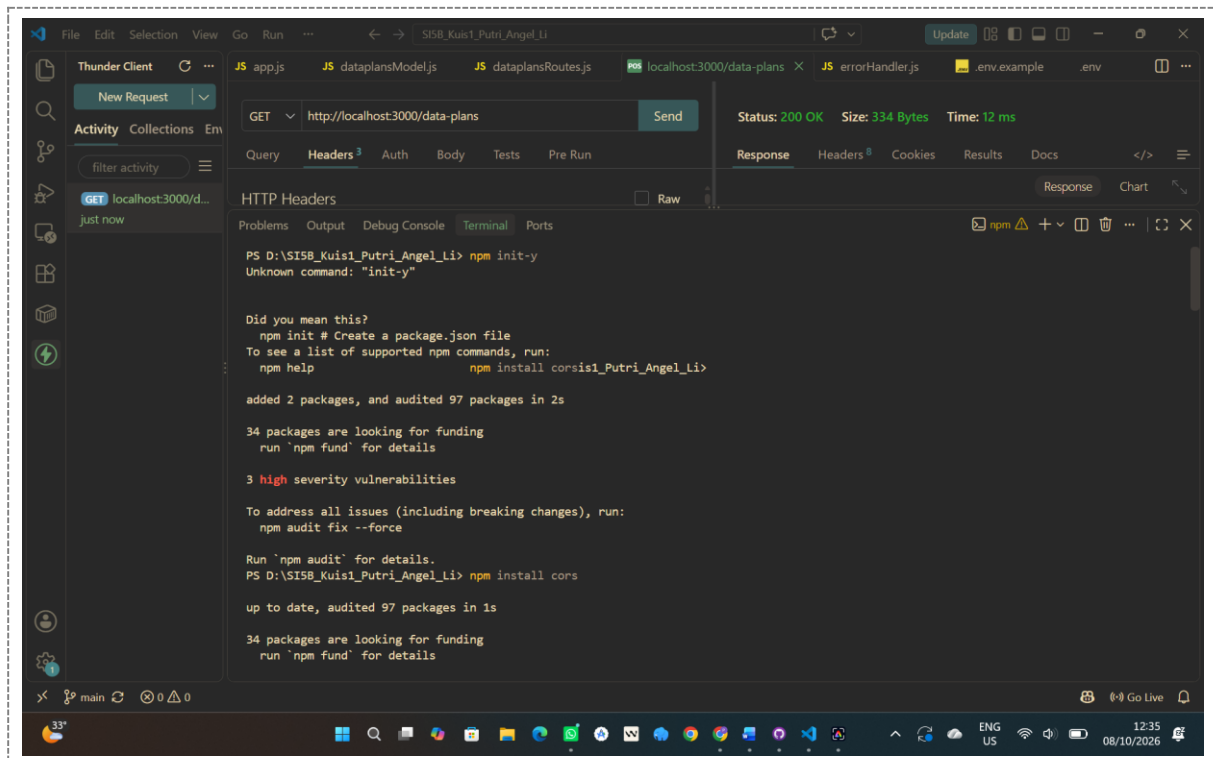
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi *express*, *cors*, dan *dotenv*, serta mengapa versi *Node.js* perlu memenuhi syarat *Express 5*.

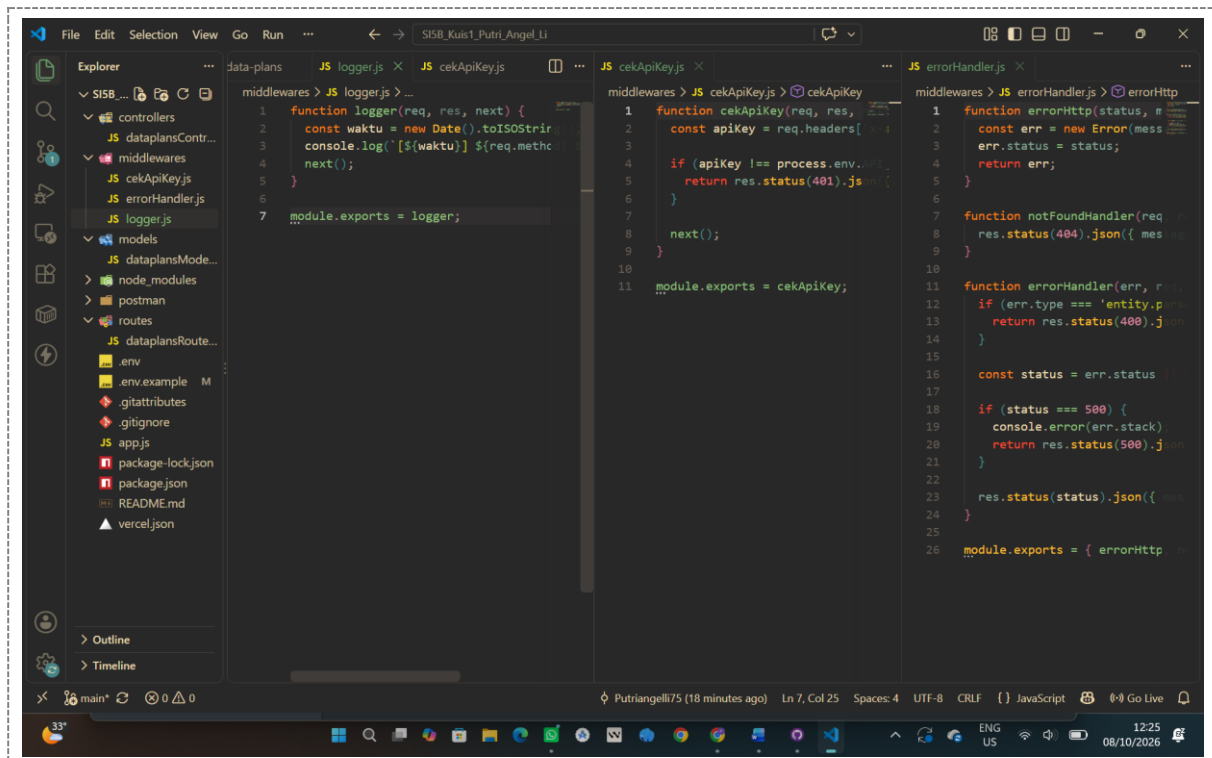
Express digunakan untuk membuat API/server, *CORS* mengatur akses dari origin berbeda, *dotenv* mengelola konfigurasi melalui environment variable, sedangkan versi *Node.js* harus sesuai dengan *Express 5* agar semua fitur dan dependency dapat berjalan dengan baik.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Buat folder controllers dengan file bernama dataplansController.js, kemudian folder middlewares dengan 3 file yaitu cekApiKey.js, errorHandler.js, dan logger.js, buat folder models dengan file dataplansModel.js, dan folder routes dengan file dataplansRoutes.js

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[dataplans]Model.js	Memisahkan data awal pada app.js ke folder models agar data pada app.js tidak banyak

Berkas	Penjelasan Anda
controllers/[dataplane]Controller.js	Menangani logika atau proses pengolahan data, seperti mengambil, menambah, mengubah, dan menghapus data data-plans, sehingga logika tidak menumpuk di file routes atau app.js.
routes/[dataplane]Routes.js	Mengatur alamat endpoint/API untuk data-plans, seperti GET, POST, PUT, dan DELETE, kemudian menghubungkannya dengan fungsi pada controller.
middlewares/logger.js	Digunakan untuk mencatat setiap request yang masuk ke server, misalnya method HTTP dan URL yang diakses, sehingga aktivitas API lebih mudah dipantau.
middlewares/cekApiKey.js	Digunakan untuk mencatat setiap request yang masuk ke server, misalnya method HTTP dan URL yang diakses, sehingga aktivitas API lebih mudah dipantau.
middlewares/errorHandler.js	Menangani error yang terjadi pada aplikasi agar server memberikan response error dalam format JSON yang lebih terstruktur dan mudah dipahami.
app.js	Merupakan file utama aplikasi Express.js yang menjalankan server, mengaktifkan middleware, menghubungkan routes, dan menentukan port server. Karena fungsi sudah dipisahkan ke model, controller, routes, dan middleware, app.js menjadi lebih sederhana dan terorganisir.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.

```

1 let dataplans = [
2   { id: 1, namaPaket: 'Internet Bul
3   { id: 2, namaPaket: 'Internet Bul
4   { id: 3, namaPaket: 'Internet Bul
5 ];
6 let nextId = 4;
7
8 function getAll(operator) {
9   if (operator) return dataplans.filter
10  return dataplans;
11 }

1 function cekApiKey(req, res,
2   const apiKey = req.headers[
3
4   if (apiKey !== process.env.API
5     return res.status(401).json({
6   }
7
8   next();
9 }
10
11 module.exports = cekApiKey;

1 function errorHandler(status, #
2   const err = new Error(mess
3   err.status = status;
4   return err;
5 }
6
7 function notFoundHandler(req,
8   res.status(404).json({ mes
9 }
10
11 function errorHandler(err, res)
  
```

```

[2026-10-08T05:22:18.307Z] GET /data-plans
PS D:\SISB_Kuis1_Putri_Angel_Li> npm run dev

> tugas1-restful-2428240091@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node app.js'
◊ injected env (4) from .env
Server berjalan di http://localhost:3000
  
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Nomor port berasal dari `.env` dan `.env.example` dengan menjalankan `npm run dev` maka akan muncul port seperti yg sudah dibuat pada `.env`. jika port sudah dipasang pada aplikasi lain maka jika menjalankan port yg sama aplikasi yg berjalan pada port sebelumnya yg akan tampil bukan aplikasi skrng

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

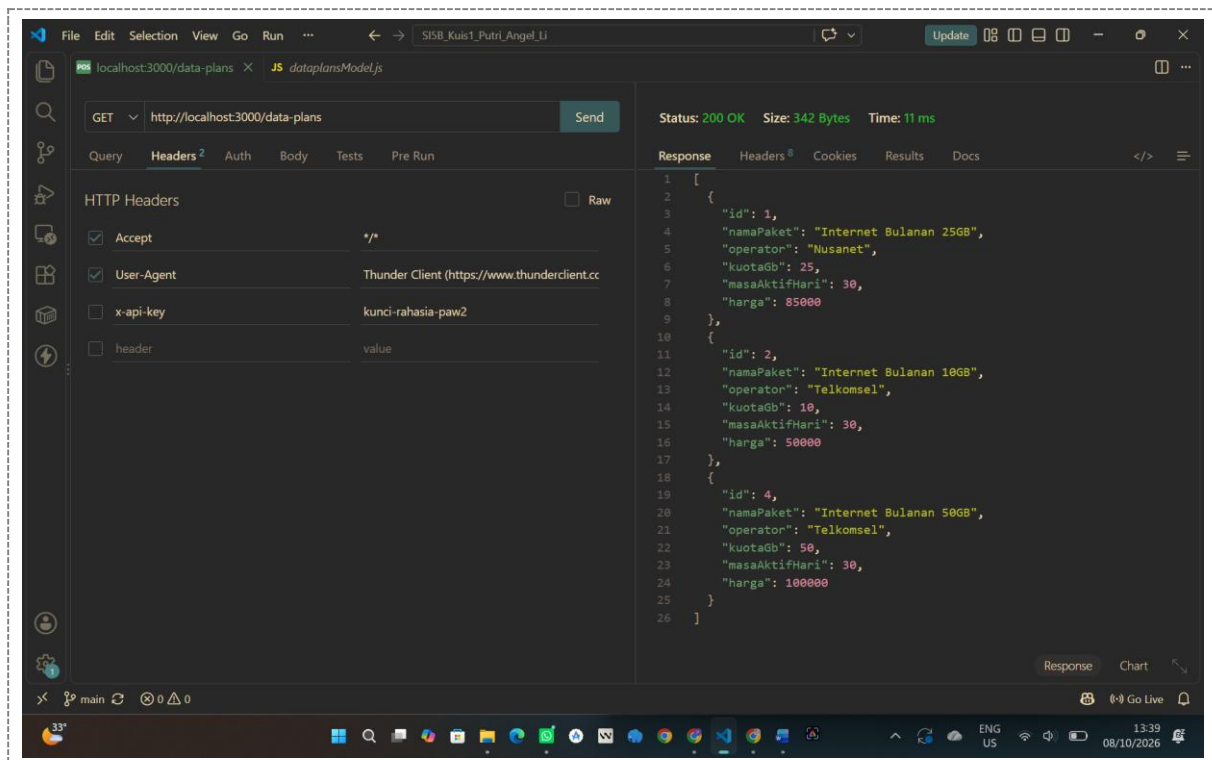
Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[data-plans])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /[dataplan]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; [{ "id": 1, "namaPaket": "Internet Bulanan 25GB", "operator": "Nusanet", "kuotaGb": 25, "masaAktifHari": 30, "harga": 85000 }, { "id": 2, "namaPaket": "Internet Bulanan 10GB", "operator": "Telkomsel", "kuotaGb": 10, "masaAktifHari": 30, "harga": 50000 }, { "id": 3, "namaPaket": "Internet Bulanan 5GB", "operator": "Axis", "kuotaGb": 5, "masaAktifHari": 30, "harga": 30000 }]

Status hasil uji	<pre>(200 OK; [{ "id": 1, "namaPaket": "Internet Bulanan 25GB", "operator": "Nusanet", "kuotaGb": 25, "masaAktifHari": 30, "harga": 85000 }, { "id": 2, "namaPaket": "Internet Bulanan 10GB", "operator": "Telkomsel", "kuotaGb": 10, "masaAktifHari": 30, "harga": 50000 }, { "id": 3, "namaPaket": "Internet Bulanan 5GB", "operator": "Axis", "kuotaGb": 5, "masaAktifHari": 30, "harga": 30000 }])</pre>
Lulus?	Ya



Gambar 4. Hasil GET semua data

Penjelasan:

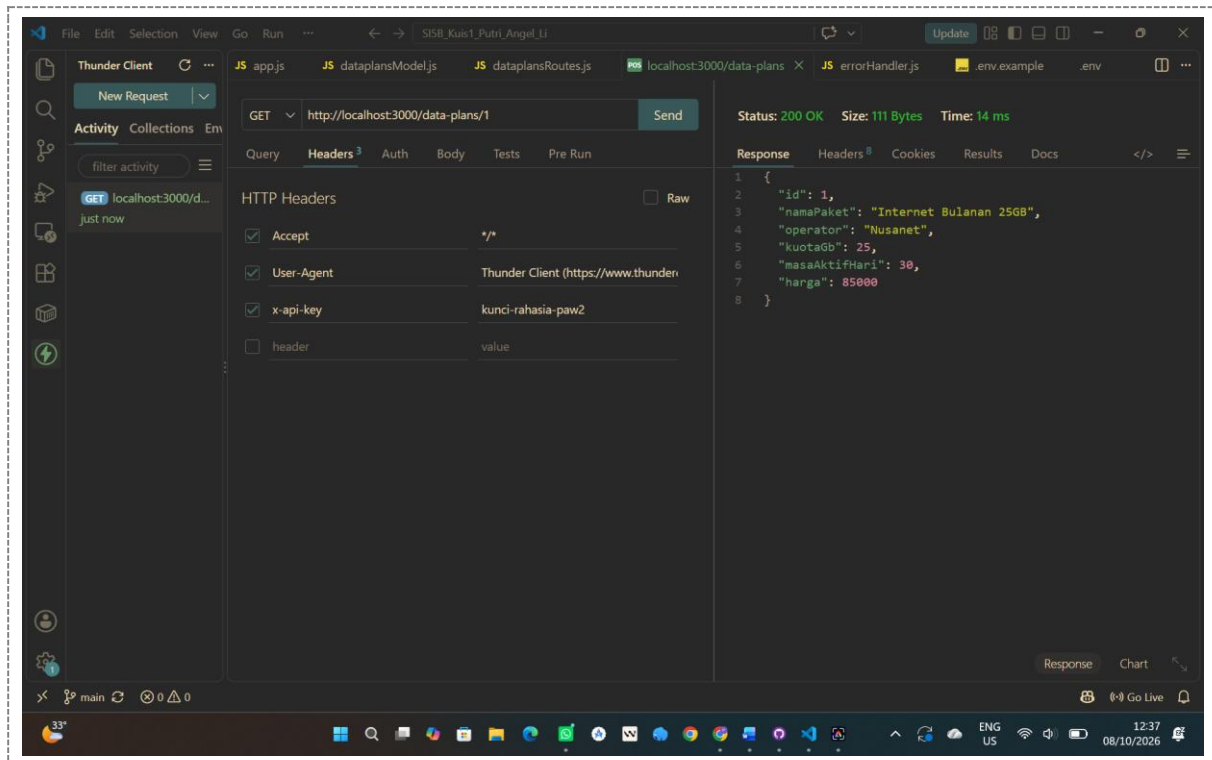
Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Kode status 200 ok yg artinya routes ini berhasil berjalan, controller yang digunakan adalah bagian `getAll` sedangkan untuk model nya juga menggunakan yang `getAll` artinya untuk menampilkan semua data awal

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /[data-plans]/[1]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; { "id": 1, "namaPaket": "Internet Bulanan 25GB", "operator": "Nusanet", "kuotaGb": 25, "masaAktifHari": 30, "harga": 85000 }.
Status hasil uji	200 OK; { "id": 1, "namaPaket": "Internet Bulanan 25GB", "operator": "Nusanet", "kuotaGb": 25, "masaAktifHari": 30, "harga": 85000 }
Lulus?	Ya



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

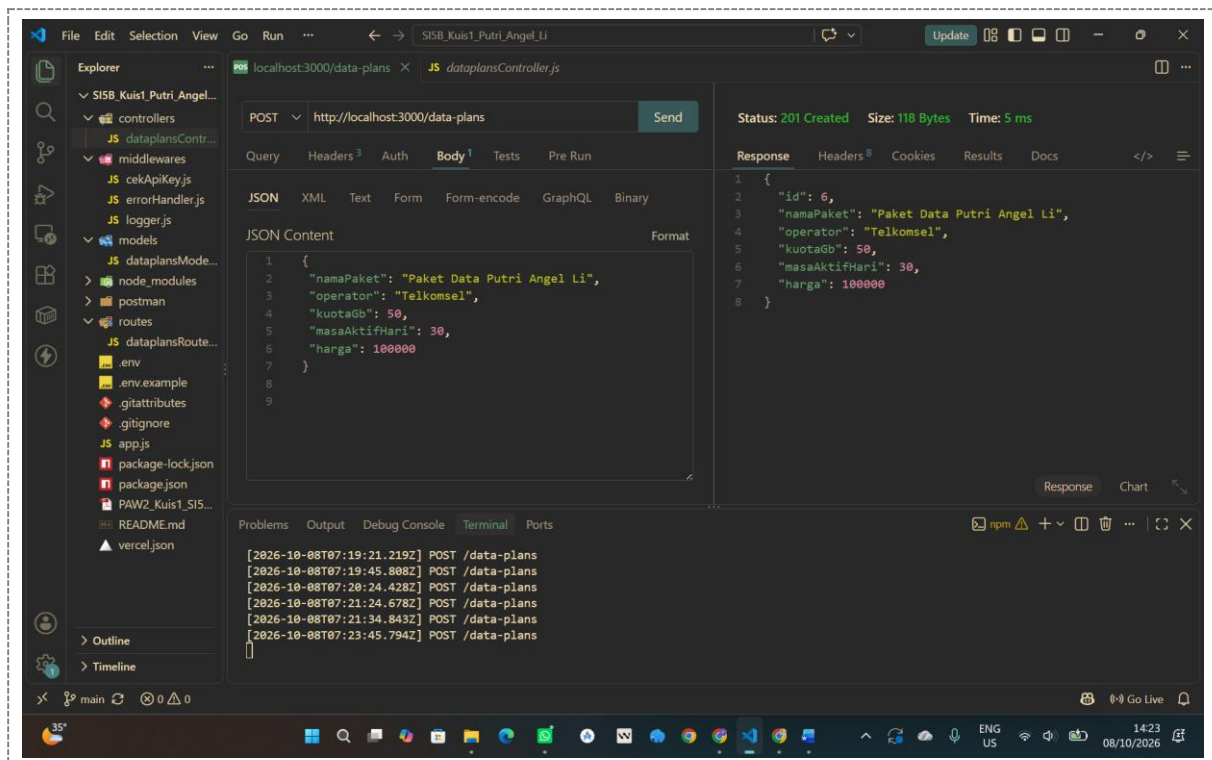
Jelaskan bagaimana ID dari alamat diambil (`req.params`) dan mengapa perlu diubah menjadi angka.

`req.params` digunakan untuk mengambil ID dari URL, sedangkan `Number()` digunakan untuk mengubah ID dari string menjadi angka agar dapat dibandingkan dengan ID pada data secara tepat.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /[data-plans]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPaket": "Paket Data Putri Angel Li", "operator": "Telkomsel", "kuotaGb": 50, "masaAktifHari": 30, "harga": 100000 }
Hasil yang diharapkan	201 Created; { "id": 4, "namaPaket": "Internet Bulanan 50GB", "operator": "Telkomsel", "kuotaGb": 50, "masaAktifHari": 30, "harga": 100000 }
Status hasil uji	201 Created
Lulus?	Ya



Gambar 6. Hasil POST data baru

Penjelasan:

Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

Header `x-api-key` digunakan sebagai kunci autentikasi sederhana untuk memastikan request memiliki API Key yang benar. Header ini diperiksa oleh middleware `cekApiKey.js`. Jika API Key sesuai dengan nilai `API_KEY` di `.env`, request diteruskan dengan `next()`. Jika tidak sesuai, request ditolak dengan status 401 Unauthorized.

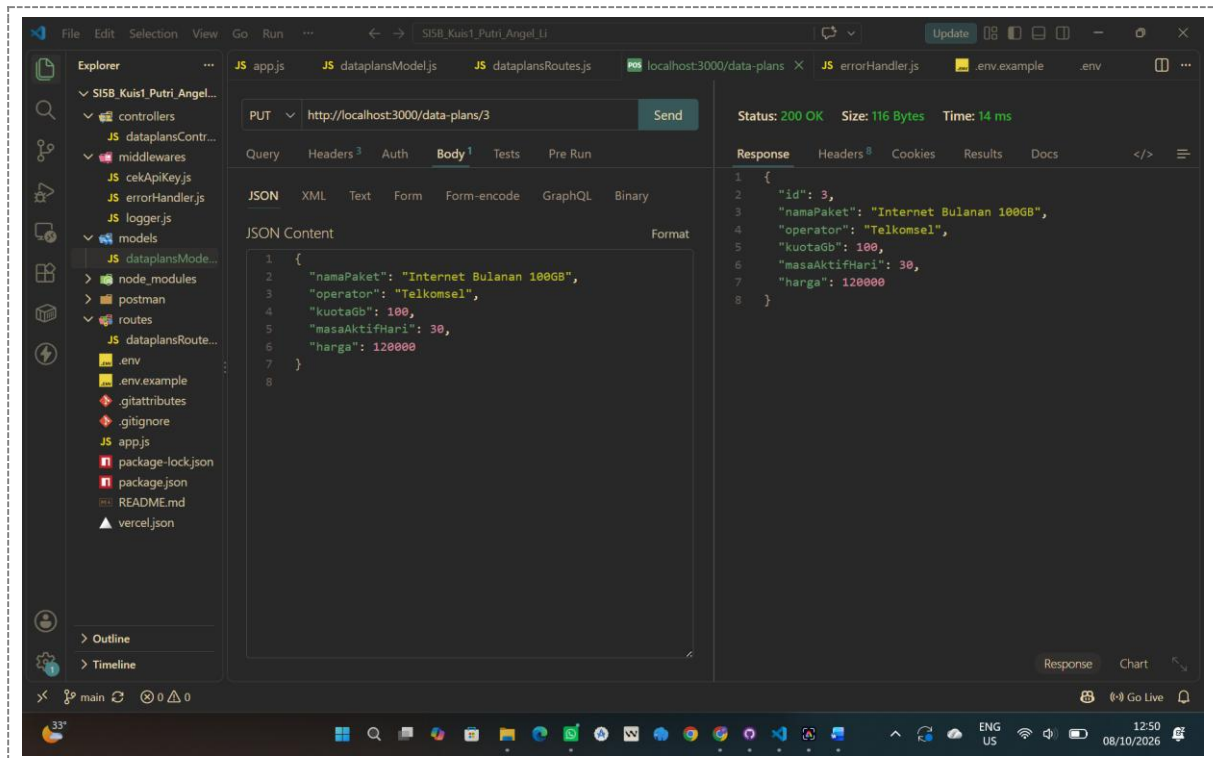
`express.json()` digunakan untuk membaca dan mengubah body request dalam format JSON menjadi objek JavaScript sehingga data dapat digunakan oleh Controller.

Status 201 Created digunakan ketika request berhasil membuat data/resource baru. Sedangkan 200 OK lebih umum digunakan untuk menunjukkan bahwa request berhasil diproses, misalnya pada operasi GET atau update data.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT /[data-plans]/3
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaPaket": "Internet Bulanan 100GB", "operator": "Telkomsel", "kuotaGb": 100, "masaAktifHari": 30, "harga": 120000 }
Hasil yang diharapkan	200 OK; { "id": 3, "namaPaket": "Internet Bulanan 100GB", "operator": "Telkomsel", "kuotaGb": 100, "masaAktifHari": 30, "harga": 120000 }
Status hasil uji	200 OK
Lulus?	[] Ya



Gambar 7. Hasil PUT mengubah data

Penjelasan:

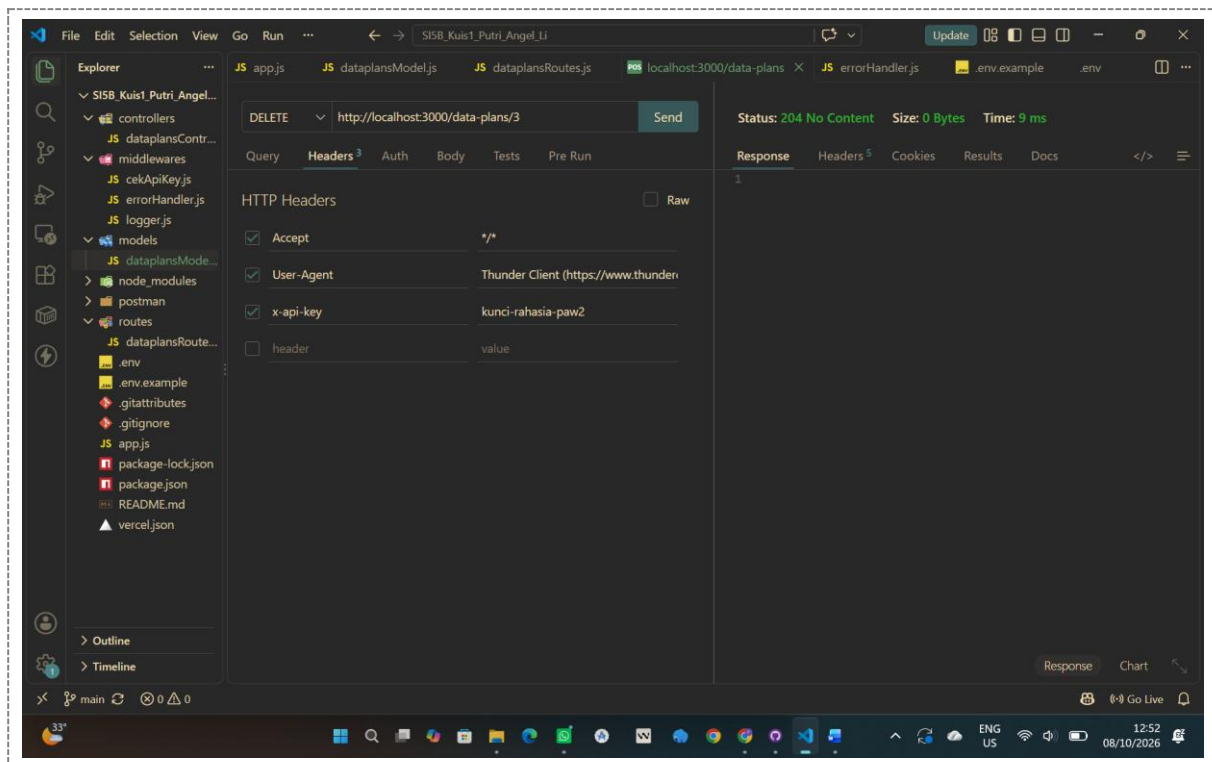
Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Id 3 pada data-plans yg awalnya `{ id: 3, namaPaket: 'Internet Bulanan 5GB', operator: 'Axis', kuotaGb: 5, masaAktifHari: 30, harga: 30000 }` diubah semua menjadi data baru

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[data-plans]/[3]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[] Ya



Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Arti status 204 adalah data berhasil dihapus dan respons tidak berisi apapun karena pada `dataplanModel.js` status message nya memang tidak ada untuk bagian delete

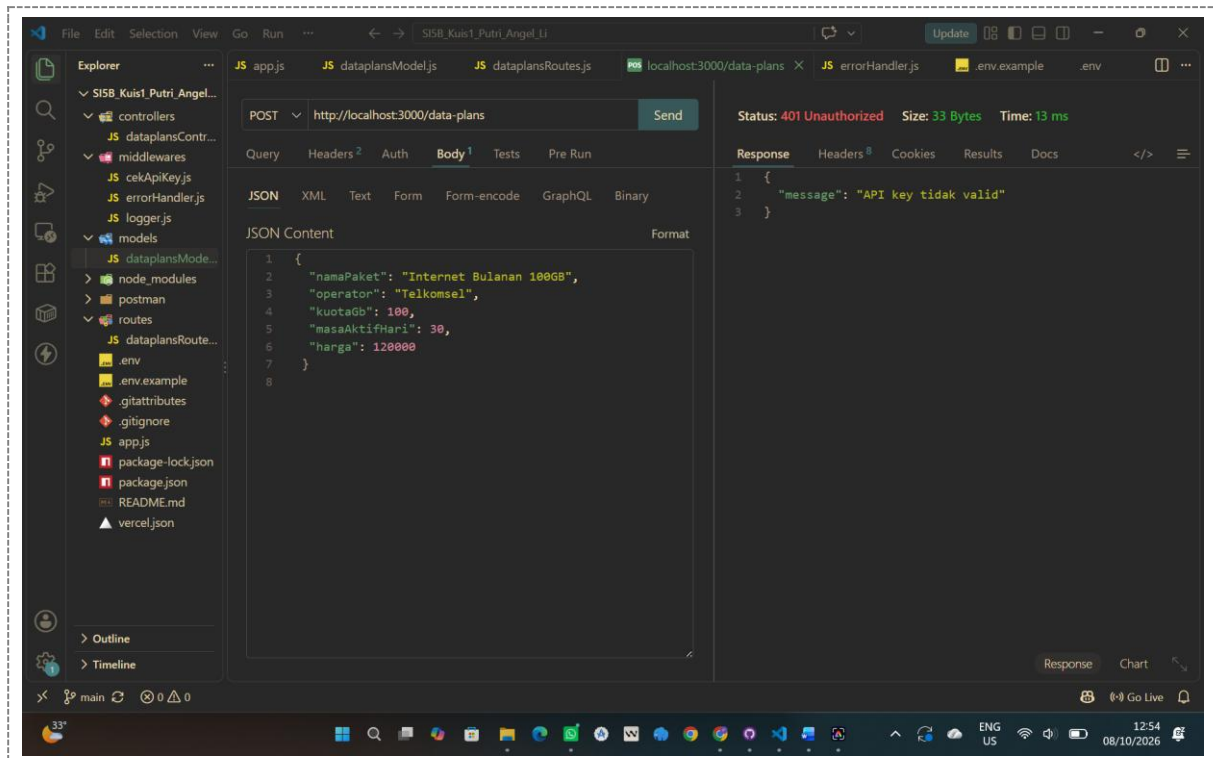
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /[data-plans] (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "namaPaket": "Internet Bulanan 100GB", "operator": "Telkomsel", "kuotaGb": 100, "masaAktifHari": 30, "harga": 120000 }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya



Gambar 9. POST tanpa API key ditolak

Penjelasan:

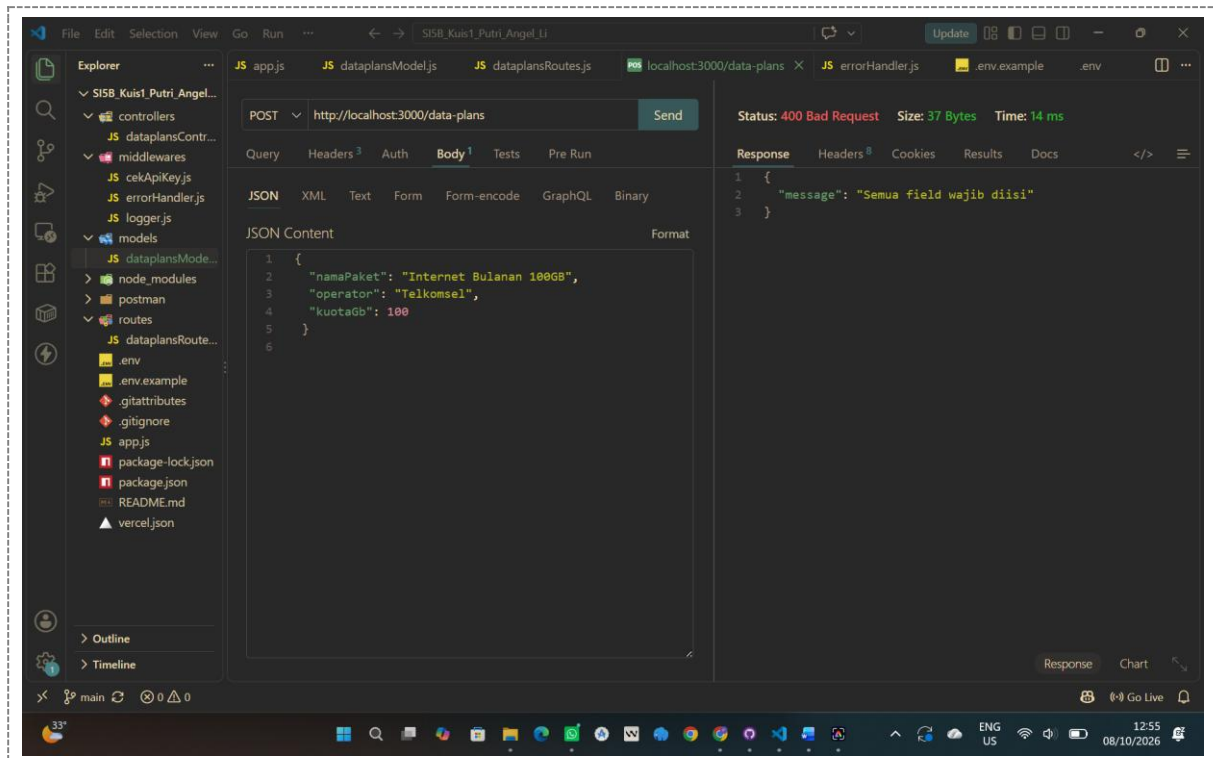
Jelaskan *middleware* mana yang menolak permintaan dan mengapa *Controller* tidak sempat dijalankan.

`cekApiKey.js` merupakan *middleware* yang menolak request karena API Key tidak valid. *Controller* tidak dijalankan karena request sudah dihentikan oleh *middleware* sebelum mencapai *controller*.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST <code>/[data-plans]</code>
Header	<code>x-api-key</code> : (nilai <code>API_KEY</code> pada <code>.env</code> Anda) <code>Content-Type</code> : <code>application/json</code>
Body	<pre>{ "namaPaket": "Internet Bulanan 100GB", "operator": "Telkomsel", "kuotaGb": 100 }</pre>
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya



Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

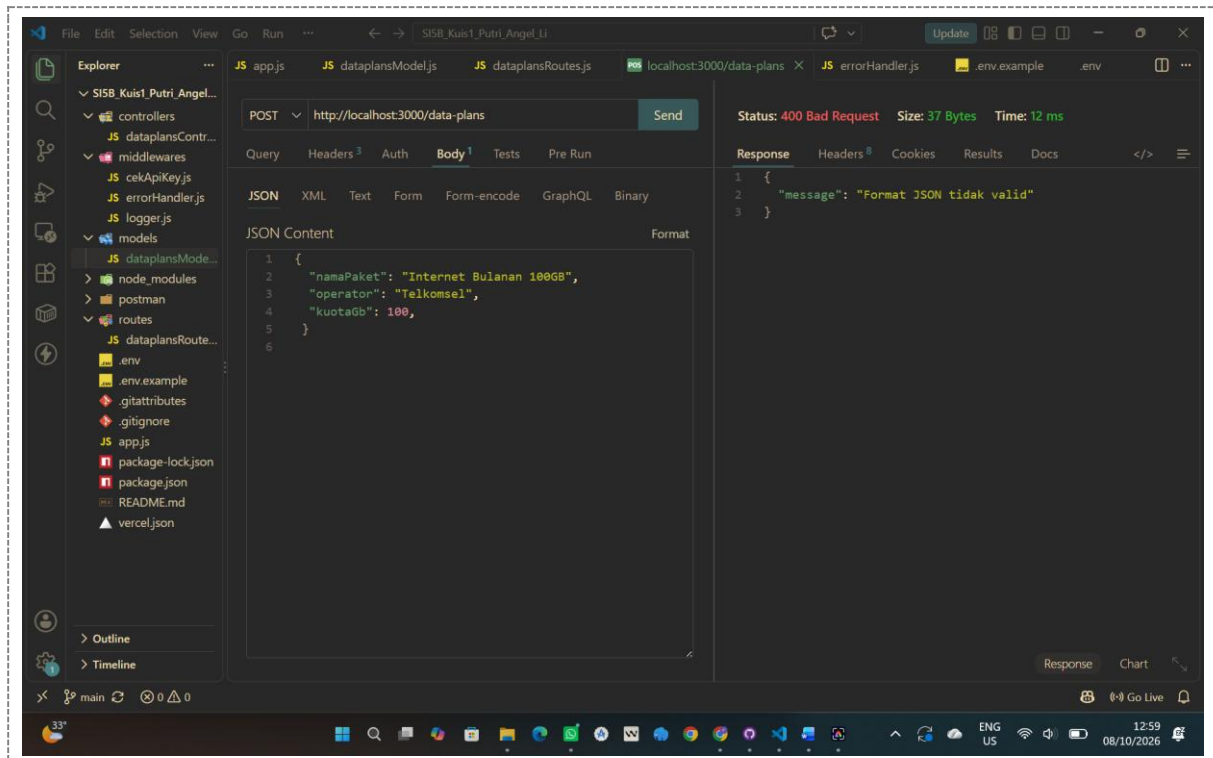
POST /data-plans → logger.js → cekApiKey.js → Routes → Controller → validasi gagal → Controller mengirim 400

Jadi, Controller yang mendeteksi data tidak lengkap dan mengirimkan respons 400, karena kesalahan terdapat pada data/request yang dikirim oleh client.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /[data-plans]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	<pre>{ "namaPaket": "Internet Bulanan 100GB", "operator": "Telkomsel", "kuotaGb": 100, }</pre>
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya



Gambar 11. POST dengan JSON rusak

Penjelasan:

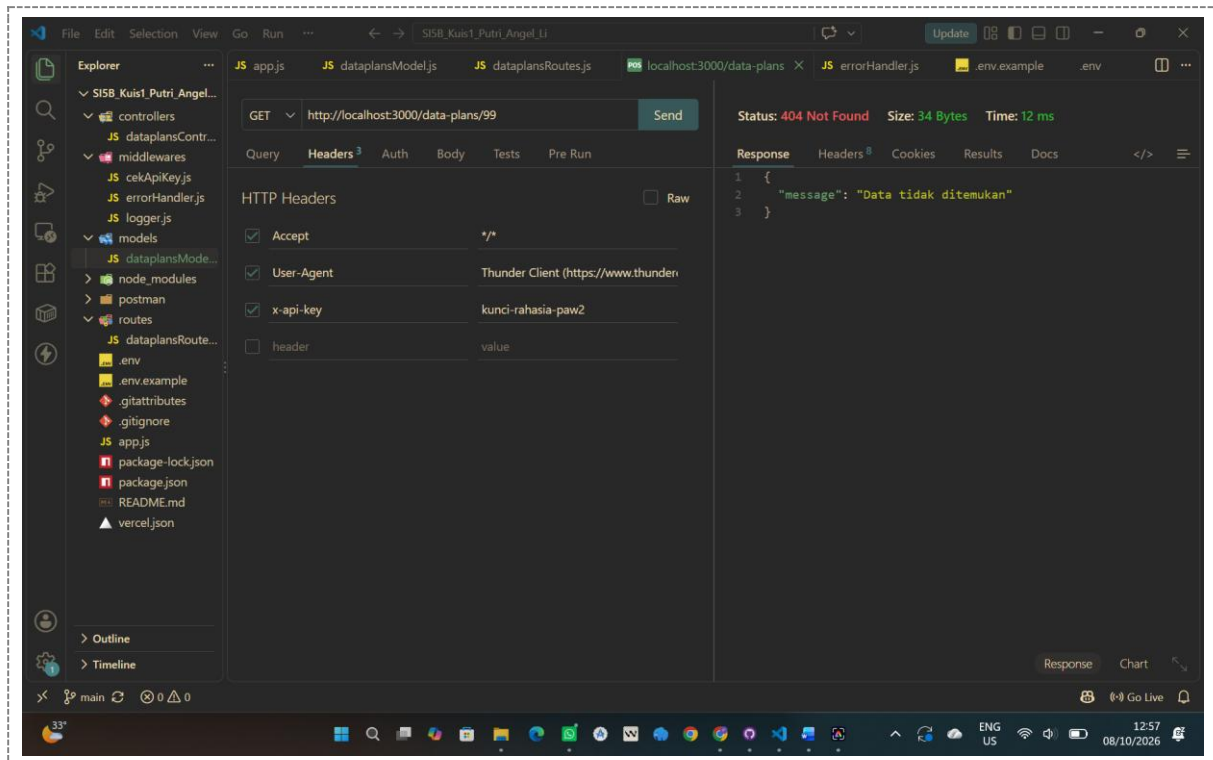
Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

Jika API Key salah, request akan berhenti di `cekApiKey.js` dan menghasilkan 401, sehingga untuk pengujian JSON rusak API Key harus benar agar request bisa melewati middleware tersebut dan mencapai `express.json()/error handler`.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET <code>/[data-plans]/99</code>
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya



Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

GET /data-plans/99 → app.js → middleware → routes → Controller → Model → data tidak ditemukan → Controller mengirim 404.

Jadi, yang membuat dan mengirim pesan 404 adalah Controller, bukan middleware atau Model.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	
T02	Menampilkan satu data berdasarkan ID	200	200	
T03	Menambah data baru	201	201	
T04	Mengubah data	200	200	
T05	Menghapus data	204	204	
N01	Menambah data tanpa API key	401	401	
N02	Menambah data dengan body tidak lengkap	400	400	
N03	Mengirim JSON yang rusak	400	400	
N04	Mengambil ID yang tidak ada	404	404	

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/Putriangelli75/SI5B_Kuis1_Putri_Angel_Li
Pesan commit terakhir	Add files via upload
Folder routes, controllers, models, middlewares ada	☐ Ya
.env tidak terunggah (ada di .gitignore)	☐ Ya
.env.example ada	☐ Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Tulis jawaban Anda di sini.

Route → Menentukan alamat/endpoint API dan menghubungkan request dengan fungsi Controller.

Contoh : `router.get('/data-plans/:id', controller.getById);`

Controller → Menangani logika proses request, seperti mengambil parameter, validasi data, dan menentukan response. Contoh : `const id = Number(req.params.id);`

Model → Menyimpan dan mengelola data data-plans, termasuk proses mencari atau memanipulasi data. Contoh : `const dataPlans = [...]`

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Tulis jawaban Anda di sini.

Keuntungannya

- Kode lebih rapi karena penanganan error tidak berulang di setiap route.
- Format response konsisten, sehingga semua error dapat menggunakan struktur JSON yang sama.
- Lebih mudah diperbaiki karena perubahan format pesan error cukup dilakukan di satu file.
- Mudah dikembangkan, misalnya nanti ingin menambahkan pencatatan error (logging) atau jenis status error lainnya.

Bukti dari pengujian

Pada pengujian GET `/data-plans/99`, ID 99 tidak ditemukan. Request kemudian menghasilkan:

```
{ "message": "Data plan tidak ditemukan" }
```

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100